Gtest For Sikernel

Gtest简介

什么是gtest: gtest测试框架是在不同平台上（Linux，Mac OS X，Windows，Cygwin，Windows CE和Symbian）为编写C++测试而生成的。它是基于xUnit架构的测试框架，支持自动发现测试，丰富的断言集，用户定义的断言。之前sikernel中测试用例未使用Gtest框架，采用的是手写main函数测试的方式，gtest相比其有如下优势：

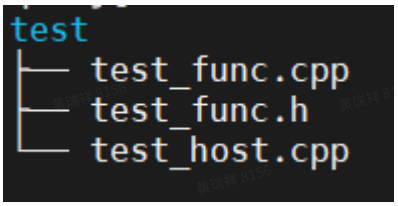
a. 减少测试代码编写量,代码整洁规范
b. 增加测试覆盖范围
c. 测试用例并行执行，减少测试时间
d. 灵活的断言机制等
e. 增加代码的鲁棒性与可测试性

由于关于gtest的知识开源的有很多，在此就不在过多赘述，有兴趣的可以参考以下链接，只针对sikernel中gtest遇到的问题加以说明。

a. https://zhuanlan.zhihu.com/p/369466622
b. https://ggdocs.cn/googletest/
c. https://github.com/google/googletest
d. https://zhuanlan.zhihu.com/p/15533978517
e. https://jishuzhan.net/article/1997252401896685570

Gtest for sikernel

针对sikernel的gtest，由于scc支持的原因需要将gtest的代码和待测试的sikernel function放置在不同的文件中，否则会出现编译时的link错误，如下图所示gtest相关代码，放置在test_host.cpp中，调用si kernel function的代码在test_func.cpp中，通过.h文件关联起来；



一个简单的gtest测试用例如下所示：通过自定义夹具定义测试的数据类型，测试数据大小，覆盖范围等（一个取巧的方式可以给类似deepseek等工具 你要测试的函数接口与测试范围等，可以生成大部分code，再次感慨ai的强大）

代码块

```cpp
#include <gtest/gtest.h>
#include <tuple>
#include <string>
#include <cstdint>
#include <limits>
#include <type_traits>
#include "test_func.h"
// ================== 类型信息辅助类 ==================
struct TypeInfo {
    std::string name;
    std::function<bool(int, int)> test_function;

    // 根据类型创建测试函数
    template<typename T>
    static TypeInfo create(const std::string& type_name) {
        return TypeInfo{
            type_name,
            [](int h, int s) { return test_func<T>(h, s); }
        };
    }
};

// ================== 通用测试类 ==================
class GenericTestFuncTest : public
::testing::TestWithParam<std::tuple<TypeInfo, int, int, bool>> {
protected:
    bool call_test_func(const TypeInfo& type_info, int H_SIZE, int SEQ_LEN) {
        return type_info.test_function(H_SIZE, SEQ_LEN);
    }
};

// ================== 通用测试用例 ==================
TEST_P(GenericTestFuncTest, AllTypeTests) {
    auto [type_info, H_SIZE, SEQ_LEN, expected] = GetParam();
    bool result = call_test_func(type_info, H_SIZE, SEQ_LEN);
    EXPECT_EQ(result, expected);
}

// ================== 测试名称生成器 ==================
std::string GenericTestName(const
::testing::TestParamInfo<std::tuple<TypeInfo, int, int, bool>>& info) {
    auto [type_info, H_SIZE, SEQ_LEN, expected] = info.param;
    return type_info.name +
           "_H" + std::to_string(H_SIZE) +
           "_S" + std::to_string(SEQ_LEN) +
           "_E" + (expected ? "T" : "F");
}
```

```
46
47  // ================== 生成所有测试配置 ==================
48  std::vector<std::tuple<TypeInfo, int, int, bool>> GenerateAllTestConfigs() {
49      // 定义所有要测试的类型
50      std::vector<TypeInfo> type_infos = {
51          TypeInfo::create<uint32_t>("uint32"),
52          //TypeInfo::create<int32_t>("int32"),
53          //TypeInfo::create<float>("float"),
54          //TypeInfo::create<uint16_t>("uint16"),
55          //TypeInfo::create<int16_t>("int16"),
56          TypeInfo::create<sifmt::bfloat16>("bfloat16"),
57          TypeInfo::create<uint8_t>("uint8"),
58          //TypeInfo::create<int8_t>("int8"),
59          //TypeInfo::create<bool>("bool"),
60      };
61
62      // 定义所有测试用例参数
63      std::vector<std::tuple<int, int, bool>> test_cases = {
64          // 正常情况
65          {256, 12, true},
66          {1024, 9, true},
67          {2, 12, true},
68          // 边界情况
69          {1, 1, true},
70      };
71
72      // 生成所有组合
73      std::vector<std::tuple<TypeInfo, int, int, bool>> all_configs;
74      for (const auto& type_info : type_infos) {
75          for (const auto& [H_SIZE, SEQ_LEN, expected] : test_cases) {
76              all_configs.emplace_back(type_info, H_SIZE, SEQ_LEN, expected);
77          }
78      }
79
80      return all_configs;
81  }
82
83  // ================== 实例化所有测试 ==================
84  INSTANTIATE_TEST_SUITE_P(
85      AllTypeParamTests,
86      GenericTestFuncTest,
87      ::testing::ValuesIn(GenerateAllTestConfigs()),
88      GenericTestName
89  );
90
91
92  // ================== 主函数 ==================
```

```
int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);

    // 打印测试配置信息
    std::cout << "Running tests for types: uint32_t, int32_t, float, "
              << "uint16_t, int16_t, uint8_t, int8_t, bool" << std::endl;

    return RUN_ALL_TESTS();
}

```

另一个更改的地方是需要在CMake中增加gtest相关的包路径等（幸运的是这块底层sdk里面已经包含有，在我们source setup.sh已经可以找到），只需要对Cmake文件做如下更改即可

代码块

```
find_package(GTest REQUIRED)
target_link_library(your_target private GTest::gtest  GTest::gtest_main
pthread)
```

其他部分保持源代码即可，一个简单的example可以参考（si-kernel/source/source_builtin/attention/where）

运行完成的测试结果如下所示，如果大家使用中遇到什么问题，可以一起愉快的交流学习成长。

```
[==========] Running 12 tests from 1 test suite.
[----------] Global test environment set-up.
[----------] 12 tests from AllTypeParamTests/GenericTestFuncTest
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint32_H256_S12_ET
[sipu_config] Warning: sipu.toml - No configuration file found, using defaults
[SIRT] Library:1.0.0.f6beb22.Release @ /share_data/sipu_sdk_debug/sipu_sdk_251215/sipu_sdk_rel/lib/libsipu.so
[sipu_config] Warning: archmodel.toml - No configuration file found, using defaults
[SIRT] Environment Shim: auto, detected Shim: swemusp
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint32_H256_S12_ET (953 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint32_H1024_S9_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint32_H1024_S9_ET (84 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint32_H2_S12_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint32_H2_S12_ET (55 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint32_H1_S1_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint32_H1_S1_ET (55 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/bfloat16_H256_S12_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/bfloat16_H256_S12_ET (72 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/bfloat16_H1024_S9_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/bfloat16_H1024_S9_ET (86 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/bfloat16_H2_S12_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/bfloat16_H2_S12_ET (55 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/bfloat16_H1_S1_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/bfloat16_H1_S1_ET (55 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint8_H256_S12_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint8_H256_S12_ET (66 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint8_H1024_S9_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint8_H1024_S9_ET (87 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint8_H2_S12_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint8_H2_S12_ET (56 ms)
[ RUN      ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint8_H1_S1_ET
[       OK ] AllTypeParamTests/GenericTestFuncTest.AllTypeTests/uint8_H1_S1_ET (54 ms)
[----------] 12 tests from AllTypeParamTests/GenericTestFuncTest (1685 ms total)

[----------] Global test environment tear-down
[==========] 12 tests from 1 test suite ran. (1685 ms total)
[  PASSED  ] 12 tests.
```